



BASIC

SELENIUM

JAVA CONCEPTS:

1. Variables
2. Data Types
3. Operators
4. Conditional Statements
5. Loops
6. Methods
7. Arrays
8. Strings
9. Classes & Objects
10. Object Class
11. Constructors
 - Default Constructor
 - Parameterized Constructor
 - Constructor Overloading
12. Encapsulation
13. Inheritance
14. Polymorphism
 - Method Overloading
 - Method Overriding
15. Abstraction
 - Abstract Classes
 - Interfaces
16. this & super
17. TypeCasting
18. File Handling
19. Exception Handling
20. Collections Framework Overview

GOALS

1. Introduction to Selenium
2. Selenium Architecture
3. Selenium WebDriver Architecture
4. Installation of Selenium
5. First Line of Code
6. WebDriver interface
7. HTML
8. SearchContext interface
9. Locators & XPath
10. Relative Locators
11. WebElement interface
12. Synchronization
13. Dropdown/List Box and Auto Suggestion
14. Actions Class
15. TakesScreenshot interface
16. JavascriptExecutor interface
17. Shadow DOM
18. Handling Frames
19. Handling Pop-Ups
20. Handling Windows

INTRODUCTION

Disadvantages of manual testing

1. Time Consuming
2. Requires more manpower
3. Tedious and monotonous job
4. Susceptible to human errors (i.e. Less accurate)

What is Automation?

- Testing a software with the help of any tool is called as automation Testing

Advantages of Automation

1. It is faster
2. Less Time consuming
3. Reduces Human Effort (Less manpower)
4. More accurate (Better Quality)

Disadvantages of Automation

1. Initial investment is High
2. 100% automation is not possible
3. Required skilled manpower

1. QTP (UFT - Unified Functional Testing)

- **Licensing: Commercial (Licensed)**
- **Language Support: VBScript only**
- Platform Support: Web, Desktop, Mobile
- OS Support: Windows, mac, linux
- Browser Support: Chrome, Firefox, Edge, IE, Safari

2. TestComplete

- **Licensing: Commercial (Licensed)**
- Language Support: 6+ langs
- Platform Support: Web, Desktop, Mobile
- OS Support: Windows (main), with limited support for macOS/Linux
- Browser Support: Chrome, Firefox, Edge, IE, Safari

3. Cypress

- Licensing: Free and open-source
- **Language Support: JavaScript**
- **Platform Support: Web applications only**

- OS Support: Windows, macOS, Linux
- Browser Support: Chrome, Edge, Firefox

4. Appium

- Licensing: Free and open-source
- Language Support: 7+ langs
- Platform Support: Mobile (iOS, Android), Web Apps (via mobile browsers)
- OS Support: Windows, macOS, Linux
- Browser Support: Chrome (Android), Safari (iOS) — via WebDriver

5. Playwright

- Licensing: Free and open-source
- Language Support: TypeScript, JavaScript, Python, C#, Java
- OS Support: Windows, macOS, Linux
- Browser Support: Chrome, Firefox, Edge

What is Selenium?

- Selenium is an automation tool, which is used to automate web based applications.
- It is a collection of libraries which consists of inbuilt classes and methods

Features of Selenium

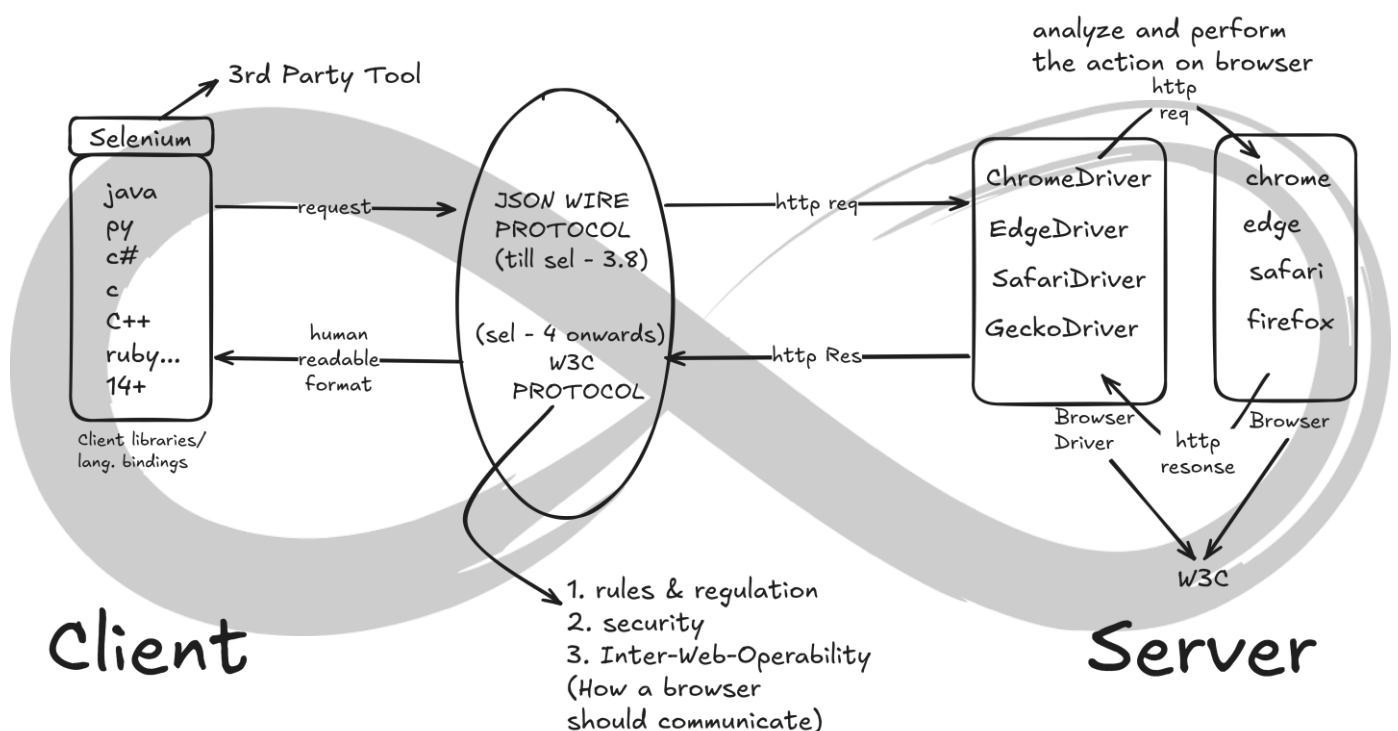
1. It is free and Open Source.
2. It supports 14+ programming Languages.
3. It supports all majorly used browsers
(e.g.: Chrome, Firefox, Edge, IE, Opera...)
4. It Supports all majorly used Operating Systems.
(e.g. Windows, Linux, mac)
5. It allows 3rd party tool integration
(e.g. TestNG, AutoIt, Cucumber...)

Disadvantages of Selenium

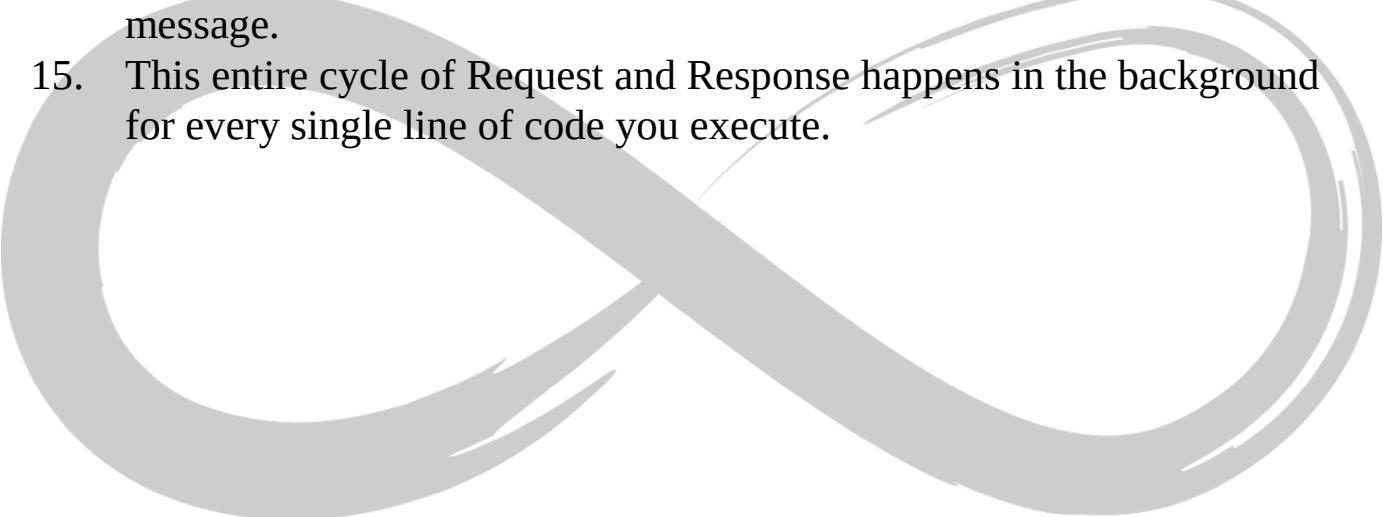
1. It supports only web based applications.
2. As it is free there is no customer Support
3. 100% automation is not possible
For example:-Captcha, OTP, Video, Audio, Animations.

SELENIUM ARCHITECTURE

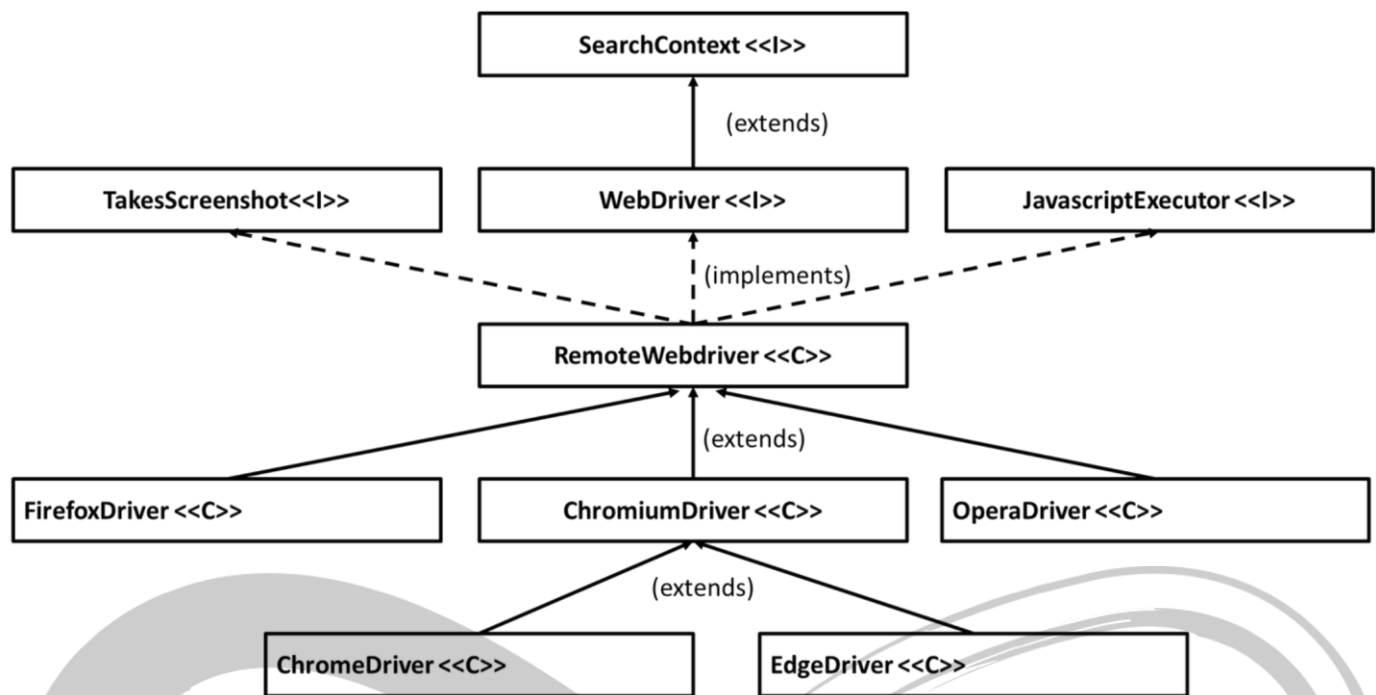
1. Selenium Architecture is also called as Client-Server Architecture which works based on a request and response cycle.
2. It all begins on the Client Side, where we use Client Libraries or Language Bindings (Java, Python, C#, etc.) to write our automation code.
3. We write these automation scripts in an Integrated Development Environment (IDE) like Eclipse, IntelliJ, or VS Code using the Selenium commands.



4. When we execute this automation script, the code tries to reach out to the browser, but the browser will not expose its internal functionality to any third-party tool for security reasons.
5. So we will interact with browser through browser driver
6. The request from client needs a protocol to carry the message, which is where the JSON Wire Protocol (in older versions) or W3C Standard (in newer versions) acts as a translating medium.
7. The request is then sent to the Browser Driver (like ChromeDriver or GeckoDriver), which acts as the Server in this architecture.

8. The Browser Driver is constantly listening for these incoming HTTP requests from the client libraries.
 9. Upon receiving the request, the Browser Driver interprets it because it is the only component that understands the internal language of its specific browser.
 10. The Browser Driver then takes control over the respective Real Browser and performs the user actions (like clicking or typing).
 11. Once the actions are performed on the browser, a response is generated regarding the status of that action.
 12. This response comes back to the Browser Driver in the form of HTTP.
 13. From the Browser Driver, the response is sent back across to the user's automation script (Client).
 14. Finally, the response is displayed in the user's console in a human-readable format, such as "Test Passed," "Element Found," or an error message.
 15. This entire cycle of Request and Response happens in the background for every single line of code you execute.
- 

SELENIUM WEBDRIVER ARCHITECTURE



SearchContext

- It is the super most interface, which provides a mechanism to search web element in a web page
- It provides 2 abstract methods
 1. findElement (By by)
 2. findElements (By by)

WebDriver

- It provides mechanism to perform browser related actions.
- It provides 11 abstract methods and 2 inherited methods.
 1. get (String url)
 2. getTitle ()
 3. getCurrentUrl ()
 4. getPageSource()
 5. close()
 6. quit()
 7. manage()
 8. navigate()
 9. getWindowHandle()
 10. getWindowHandles()
 11. switchTo()

TakesScreenshot

- It provides mechanism to take screenshot of entire web page
- It provides 1 abstract method
getScreenshotAs()

JavascriptExecutor

- It provides mechanism to write a script in browser understandable language (JavaScript)
- It provides 2 abstract Methods
 1. executeScript()
 2. executeAsyncScript()

Note:

All the implementation of these methods are given in RemoteWebDriver class.

Respective Browser specified classes extends RemoteWebDriver Class.

SELENIUM INSTALLATION

Step 1: Open the New Project Wizard

- In Eclipse, go to the top menu and click **File > New > Project...**
- In the wizard window that appears, type **Maven** in the search bar.
- Select **Maven Project** from the list and click **Next**.

Step 2: Create a Simple Project

- Check the box that says "**Create a simple project (skip archetype selection)**".
- Click **Next**.

Step 3: Enter the Project Details

- **Group Id:** Type org.automationwithpiyushsir (Think of this as the unique identifier or domain for our organization).
- **Artifact Id:** Type basic-selenium-batch_code (This will be the actual name of your project folder).
- Click **Finish**.
- *Important Note: Make sure your computer is connected to the internet so Maven can download its initial configuration files.*

Step 4: Open Your Configuration File

- Once your project is generated, look at the **Project Explorer** panel on the left side of your screen.
- Double-click on the pom.xml file to open it. This is the heart of your Maven project where we tell it to bring in Selenium.

Step 5: Add the Selenium Dependency

- Scroll down inside the pom.xml file. Find the `</version>` tag and the `</project>` tag at the very bottom.
- Create a new section by typing `<dependencies>` right between them.
- Inside this new section, paste the code for the latest stable version of Selenium Java (currently 4.45.0). Your final code block should look exactly like this:

```
<dependencies>
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.43.0</version>
</dependency>
</dependencies>
```

- **Save the file** (Ctrl + S on Windows or Cmd + S on Mac). You will see a progress bar in the bottom right corner of Eclipse as Maven connects to the internet and automatically downloads all the necessary Selenium files into your project.

FIRST LINE OF CODE

```
ChromeDriver driver1 = new ChromeDriver();  
EdgeDriver driver2 = new EdgeDriver();  
FirefoxDriver driver3 = new FirefoxDriver();
```

Upcasting the Browser driver

```
WebDriver driver= new ChromeDriver();  
        driver = new EdgeDriver();  
        driver = new FirefoxDriver();
```

Here,

- WebDriver is an Interface
- driver is a reference variable.
- New is a keyword which will create random memory space in heap area and
- ChromeDriver() is constructor calling statement.

Where, Constructor has 3 jobs,

1. Load, Register and Reinitialize the non-static members
2. Start the server
3. Launch an empty Browser

Advantages of Upcasting

1. We can achieve runtime polymorphism
2. We can achieve Cross Browser Testing
3. We can achieve code optimization

1. get (String url)

- It is used to navigate to an application specified by the URL
- URL should be fully qualified path
- Return Type is void

2. getTitle ()

- It is used to capture the title of the web page
- It is used for Validation Purpose.
- Return type is String.

3. **getCurrentUrl ()**

- It is used to capture the URL of the web page.
- Return type is String.

4. **getPageSource ()**

- It is used to capture the source code (HTML, CSS, JS) of the web page
- Return type is String.

5. **close ()**

- It is used to close the browser where the driver control is present (focused page)
- It won't stop the server.
- Return type is void.

6. **quit ()**

- It is used to close all the browser windows that are opened by Selenium
- It will stop the server
- Return type is void.

7. **manage ()**

- It used for managing purposes.
- Return type is Options which is an interface in Selenium
- In Options interface we have:
 - a) window ()
 - b) timeouts ()
 - c) cookies methods

a) window ()

Return type is Window, which is an Interface in Selenium.

In Window, we have-

1. maximize ()
2. minimize ()
3. full screen ()
4. getSize ()
5. setSize ()

6. `getPosition ()`

7. `setPosition ()`

I.maximize()

It is used to maximize the browser window

E.g.: - `driver.manage().window().maximize();`

II.minimize()

It is used to minimize the browser window

E.g.: - `driver.manage().window().minimize();`

III.fullscreen()

It is used to make browser fullscreen

eg:- `driver.manage().window().fullscreen();`

IV.getSize()

It is used to get the height and width of the web page.

Return type is `Dimension`, which is a class in Selenium.

Eg:- `Dimension ref= driver.manage().window().getSize();`

V.setSize(Dimension targetSize)

It is used to set the size of browser.

Eg:-

`driver.manage().window().setSize(new Dimension(width, height));`

VI.getPosition()

It is used to get the position of the current window, relative to the upper left corner of the screen. Return type is `Point`, which is a class in Selenium.

E.g.:-

`Point ref= driver.manage().window().getPosition();`

VII.setPosition(Point targetPosition)

It is used to set the position of the current window, relative to the upper left corner of the screen.

Eg:-

```
driver.manage().window().setPosition(new Point(x, y))
```

b)timeOuts()

c)cookies()

A cookie is a small piece of data that is sent from a website and stored in your computer. Cookies are mostly used to recognize the user and load the stored information.

```
driver.manage().getCookies(); → Set<Cookie>
```

```
driver.manage().getCookieNamed(null); → Cookie
```

```
driver.manage().addCookie(null); → void
```

```
driver.manage().deleteCookie(null); → void
```

```
driver.manage().deleteCookieNamed(null); → void
```

```
driver.manage().deleteAllCookies(); → void
```

How Cookies Work:

When a User Visits a Website: The website's server sends a cookie to the user's browser. The browser stores this cookie locally on the user's device.

On Subsequent Visits: The browser sends the stored cookie back to the website's server.

The server uses the information in the cookie to customize the user's experience.

Common Uses of Cookies:

Authentication: Keeping users logged in.

Personalization: Remembering user preferences like language or theme.

Analytics: Tracking user behavior to improve the website.

Advertising: Delivering targeted ads based on browsing history.

Security and Privacy Concerns:

Cookies can be used to track users across websites, raising privacy issues.

Mismanagement of cookies can lead to vulnerabilities like session hijacking.

8. navigate()

- It is used to navigate within the application
- Return type is Navigation.
 - which is an Interface in Selenium
- In Navigation Interface, we have-
 - i. to(String url)
 - ii. to(URL url)
 - iii. forward()
 - iv. back()
 - v. refresh()

i. to(String url)

- It is another way of opening an URL or application without using get().
- eg:- `driver.navigate().to("url");`

ii. to(URL url)

- It is also used to navigate to application without using get().
- Here, we create an URL object
- eg:- `driver.navigate().to(new URL("url"));`

iii. forward()

- It is used to go forward or to next page .
- eg:- `driver.navigate().forward();`

iv. back()

- It is used to go to previous page.
- eg:- `driver.navigate().back();`

v. refresh()

- It is used to refresh the web page.
- eg:- `driver.navigate().refresh();`

9. **getWindowHandle()**

- It is used to capture the window id of a web page, where the driver control is present.
- Return type is String.

eg:- `driver.getWindowHandle();`

10. **getWindowHandles()**

- It is used to capture all the window ids of a web page those are opened by Selenium.
- Return type is Set<String>.
- eg:- `driver.getWindowHandles();`

11. **switchTo()**

- It is used to switch the driver control.
- Return type is TargetLocator type Object.
 - which is an interface in Selenium.
- In TargetLocator Interface, we have-
 - a) `window()`
 - b) `frame()`
 - c) `alert()`

eg:-

- `driver.switchTo().window();`
- `driver.switchTo().frame();`
- `driver.switchTo().alert();`

close()	quit()
Only closes current Window	Closes both parent & child Windows
It will not stop the Server	It will Stop the Server
Return type is void	Return type is void
It should be the last line of code	It should be the last line of code

get()	navigate()
It is used to open an application/ web page	It is used to open an application/ web page
It will wait until the web page is fully loaded	It will not wait until the web page is loaded.
Here, we can only open the application	Here, we can open and perform other actions like forward, back and refresh
Return type is void	Return type is Navigation

getWindowHandle()	getWindowHandles()
It is used to capture the window id of parent window page	It is used to capture the all window ids of web page (both parent & child windows)
Return type is String	Return type is Set<String>

HTML

- HTML stands for “HyperText Markup Language”
- It is used to construct the structure of a web page.
- It is made up of tags.
- Tags:- Predefined function to perform certain set of actions.

CORE STRUCTURE OF HTML

```
<html>
  <head>
    <title> Selenium </title>
  </head>
  <body>
    -- View port Area --
  </body>
</html>
```

Tags

Tags are 2 types

1. paired tag
2. unpaired tag/ self-closing tag

Paired Tag: -

- Tag which has both opening and closing tags is considered as paired tag
- They can have child tags, plain text and attributes.

eg:- <html> </html>

<p> </p>

<head> </head>, etc.

2. Unpaired tag:-

- Unpaired tags are opened tag and do not have to be closed.
- They are also called as self closing tags

eg:-
, , <input>, etc.

3 Golden rule of HTML

1. The word which is enclosed between < and > symbol is called as Tagname.
2. The pair of word which is surrounded by = symbol is called as Attribute name and Attribute value pair
3. The word which is enclosed between > and < symbol is called as Visible Text

Path is used to find elements in HTML code

There are 2 types-

1. Absolute Path
2. Relative Path

Absolute Path

Starting from HTML tag to the desired element or the required element is called an Absolute Path

It starts with “/”.

Syntax:- /html/body/tag-name/child tag-name

2. Relative Path

Directly jump to the desired element.

It starts with “//”.

Syntax:- //tag-name/child tag-name

Note:-

/ Represents immediate child

// Represents descendant

SEARCHCONTEXT INTERFACE

1. findElement(By by)

1. It helps us to locate one element at a time.
2. It takes an argument called “By”, where we should pass By type object reference.
3. Return type is WebElement.

2. findElements(By by)

1. It helps us to locate multiple elements at a time.
2. It takes an argument called “By”, where we should pass By type object reference.
3. Return type is List<WebElement>.

By :-

- By is an abstract class in Selenium.
- It Provides 8 Static Methods
 1. id()
 2. name()
 3. linkText()
 4. partialLinkText()
 5. className()
 6. tagName()
 7. cssSelector()
 8. xpath()
- All these methods accept “String” as an argument.
- Return type is By type object reference.
- All these methods are considered as Locators.

Absolute Path	Relative Path
We have to traverse from root tag to desired element	Can directly jump to required element
It starts with “/”.	It starts with “//”.
slower	faster
Long path	Optimized path

findElement()	findElements()
Locate one element at a time	Locate multiple elements at a time
It returns first matching element	It returns all the matching records
Return type is WebElement	Return type is List<WebElement>
If element not found, we get NoSuchElementException.	If elements not found, we get empty List or List with size 0.
It takes By as an argument	It takes By as an argument



LOCATORS & XPATH

- It is used to identify/locate the web elements in the web page.

Types of Locators-

1. id
2. name
3. linkText
4. partialLinkText
5. className
6. tagName
7. cssSelector
8. Xpath

Note:

- All these methods are present in “By” class and they are static methods which will take String as an argument.

Eg:- `By.id(“String”)`

`By.name(“String”)`

1. id()

- It is used to identify an element using “id” attribute present in HTML code.
- It is given top-most priority.
- It is faster as id is unique.

2. name()

- It is used to identify an element using “name” attribute
- It is given second Priority.

3. linkText()

- It is used to identify an element using text which is present between anchor tags (`<a> text </a>`)
- It is given third priority.

4. partialLinkText()

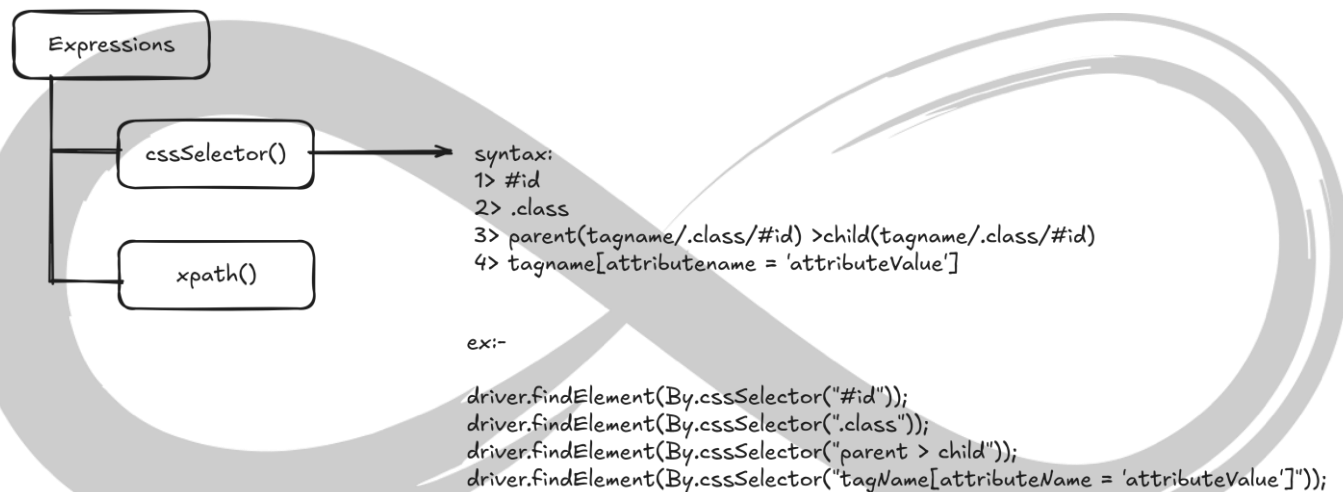
- We go for partialLinkText only when-
- Text between anchor tags is very lengthy
- Text between anchor tags is partially dynamic
- When the text has blank spaces at the beginning or at the end of a text

5. className()

- It is used to identify an element using “class” attribute present in HTML code.
- It is least recommended, because it is dynamic in nature.

6. tagName()

- It is used to identify an element using tag-name.
- It is not recommended, because multiple elements will be developed using same tag-name.
- When we have to find total number of elements present, then we can make use of this locator.



7. cssSelector()

- It is used to identify an element using following syntax.
- Syntax:- `tagname[attribute name='attribute value']`
- Tag must contain at least 1 attribute.
- It is unidirectional.

8. xpath()

- It is one of the relative paths.
- It is multidirectional.

Syntax:- `//tagname[@attributeName='attributeValue']`

- a) xpath by Attribute
- b) xpath by Text Function
- c) xpath by Contains(), starts-with(), normalize-space()
- d) xpath by Multiple Attributes
- e) xpath by Surroundings
- f) xpath by Axes
- g) xpath by Indexing

a) xpath by Attribute

- Always choose the attribute name that appears only once.
- Syntax:- `//tag-name[@attribute-name='attribute-value']`

b) xpath by Text Function

- Also known as xpath by visible text.
- Visible text:- A text which is present between any tags.
- Syntax:- `//tag-name[text()='text-value']`

c) xpath by contains(), starts-with(), normalize-space()

- If attribute value or text value is very lengthy, we just make use of part of that value to locate entire element.
- Syntax:-
`//tag-name[contains(@attribute-name,'attribute-value')]`
`//tag-name[contains(text(),'text-value')]`
`//tag-name[normalize-space()='Login']`
`//tag-name[starts-with(@id,'user')]`

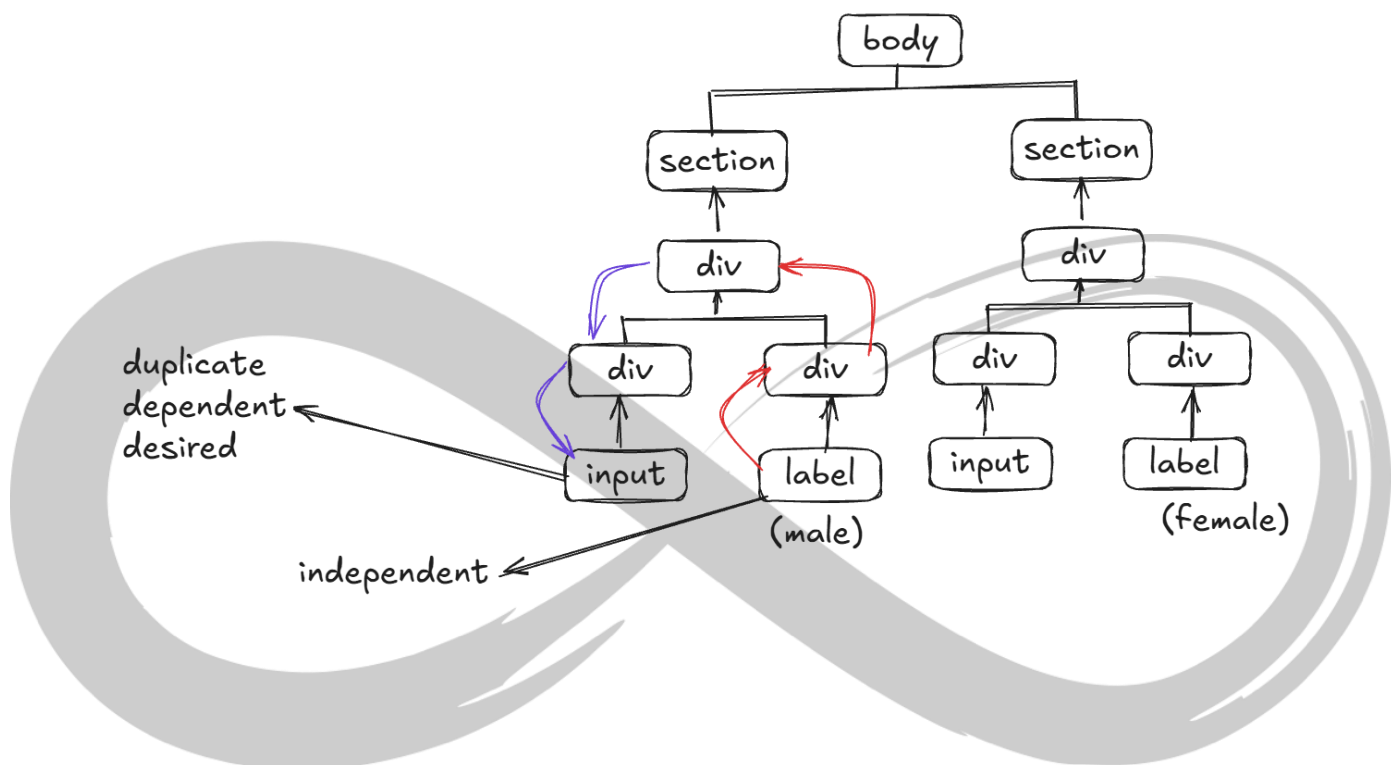
d) xpath by Multiple Attributes

- By making use of more than 1 attribute value or text value we can locate web element, with the help of logical operators(and, or)
- Syntax:-

```
//tag-name[@attribute-name='attribute-value' and @attribute-
name='attribute-value' ]
```

OR

```
//tag-name[@attribute-name='attribute-value' or text()='text-value']
```



e) xpath by Surroundings

It is also called as dependent and independent xpath.

To locate dependent and independent element, we need to follow 4 steps.

Step 1: Identify dependent and independent element

Step 2: Write xpath for independent element

Step 3: Go to common parent element by using “/..”

Step 4: Come back to Desired element

f) xpath by Axes

- some of the axis-names are-
 1. following-sibling
 2. preceding-sibling
 3. ancestor
 4. descendant
 5. parent
 6. child
- Syntax:-
`//tag-name[@attribute-name='attribute-value']/axis-name :: tag-name`
OR
`//tag-name[text()='text-value']/axis-name :: tag-name`

g) xpath by Indexing

- This is not preferable.
- It should be the last option when none of the xpath or locators are not working, we can make use of it.
- Syntax :
`(xpath) [index]`

Difference btw cssSelector vs xpath

Sr.	cssSelector	xpath
1	We cannot use visible text	We can use visible text using text()
3	It uses CSS-like Syntax	It uses Path-like Syntax
4	Syntax: tagname[AN='AV']	Syntax: //tagname[@attribute-name='attribute-value'] OR //tagname[text()='text-value']
5	Concept of Traversing is not there (“/.”) (Unidirectional)	Concept of Traversing is there (“/.”) (Bidirectional)
6	Cannot use Logical Operators (and, or, not)	Can use Logical Operators (and, or, not)
7	We cannot use Indexing	Indexing can be used

RELATIVE LOCATORS

Introduction

- Relative Locators were introduced in Selenium 4 to locate web elements based on their position relative to another element.
- Traditional locators:
`driver.findElement(By.id("username"));`
- Relative Locators:
`driver.findElement(with(By.tagName("input")).above(passwordField));`

Why Relative Locators?

Problems:

- Dynamic IDs
- Dynamic Names
- No unique locator available
- Modern UI applications

Solution:

- Use nearby stable elements.

Available Relative Locators

1. `above()`
2. `below()`
3. `toLeftOf()`
4. `toRightOf()`
5. `near()`

Syntax

```
import static org.openqa.selenium.support.locators.RelativeLocator.with;
```

```
WebElement element = driver.findElement(  
                                with(By.tagName("input"))  
                                .above(passwordTextbox)  
                                );
```

Advantages

- Easy to maintain
- Useful for dynamic applications
- Readable code

Limitations

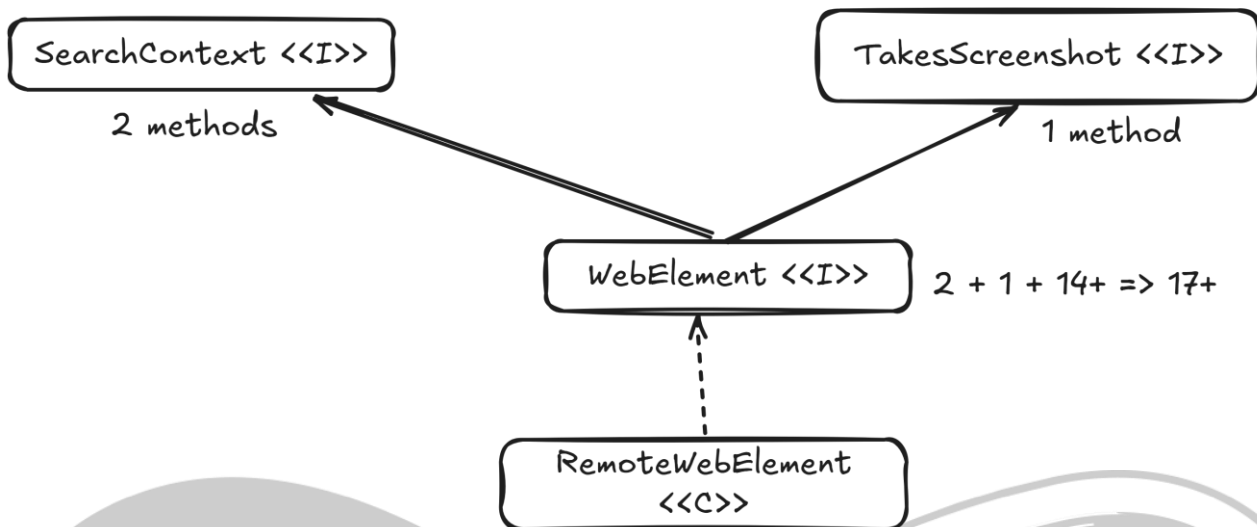
- Performance slower than ID
- Position changes can break tests

Interview Questions

1. What are Relative Locators?
2. List all Relative Locators.
3. Difference between normal locator and Relative Locator.



WebElement Architecture



SearchContext

- It has 2 abstract methods
 1. findElement(By by)
 2. findElements(By by)

WebElement

- It is an interface which provides mechanism to perform actions on a web element.
- It has 14 inbuilt methods and 3 inherited methods (all abstract methods)
- WebElement Interface has several abstract methods which can be categorized into 3
 1. Actions
 2. Getters
 3. Verifications/Validations

1. Actions

These are the action methods which are used to perform actions on the web element.

1. click()
2. submit()
3. clear()
4. sendKeys()

i. click() :

- It is used to perform click actions on an element.
- Return type is “Void”.

ii. clear()

- It is used to clear the content which is entered inside the text fields.
- Return type is “Void”.

iii. submit() :

- It is similar to click() and used to perform click action on an element.
- It must satisfy the following conditions:-
- Element should contain [type=“submit”] attribute
- Element should be child of form tag.
- Return type is “Void”

iv. sendkeys() :

- It is used to enter the data inside text field
- Return type is void.

2. Getters

- These are the getters methods which are used to return a particular value related to the web element.

1. `getTagName()`
2. `getAttribute()`
3. `getCssValue()`
4. `getText()`
5. `getSize()`
6. `getLocation()`
7. `getRect()`

i. `getText()` :

- This method is used to fetch the text on the element.
- Return type is “String”.
- It is used for validation purpose.

ii. `getAttribute ()` :

- This method is used to fetch the attribute value for the specified attribute name from the element node
- It takes attribute name in the String format as an argument.
- Return type is “String”.

iii. `getCssValue()` :

- This method is used to fetch the properties/ css value of an element on the webpage(i.e., color, font-size, background color, etc.)
- It accepts String type as an argument.
- Pass the attribute name inside the method and it will return the respective attribute value.
- Return type is “String”.

iv. `getTagname ()` :

- This method is used to fetch the tagname of the element.
- Return type is “String”.

v. getSize() :

- This method is used to fetch the size of an element on the webpage.
- It returns the height and width of the element in pixel.
- Return type is “Dimension”.
- Dimension is a class which contains the following methods
 1. getHeight() : This is used to fetch the height of an element and its Return type is “int”.
 2. getWidth() : This is used to fetch the width of an element and its Return type is “int”.

vi. getLocation() :

- This method is used to fetch the location of an element on the webpage in the form of x and y coordinates in pixels.
- Return type is “Point”.
- Point is a class which contains the following methods
 1. getX() : To fetch the X coordinates of an element and its Return type is “int”.
 2. getY() : To fetch the Y coordinates of an element and its Return type is “int”.

vii. getRect() :

- This method is used to fetch the location and size of an element on the webpage.
- Return type is “Rectangle” type object.
- Rectangle is class which contains the following methods
 1. getDimension()
 2. getHeight()
 3. getWidth()
 4. getPoint()
 5. getX()
 6. getY()

3. Verification

- These are the verification methods which are used to verify the web elements
 1. isDisplayed()
 2. isEnabled()
 3. isSelected()
- All the 3 methods are also called as “Boolean Methods”.

i. isDisplayed() :

- This method is used to check if an element is displayed on the webpage or not.
- Return type is “boolean”.

ii. isEnabled() :

- This method is used to check if an element is enabled on the webpage or not.
- To use this, button tag should contain “disabled” attribute.
- Return type is “boolean”.

iii. isSelected() :

- This method is used to check if an element is selected on the webpage or not.
- Return type is “boolean”.

SYNCHRONIZATION

- The process of matching selenium test script speed with the speed of web application is called as Synchronization.
- When the page loads on browser, various web elements on it may load at different time intervals.
- If the selenium script speed is faster than the application speed, it throws an exception (i.e., '**NoSuchElementException**' or '**ElementNotVisibleException**')
- So, we make use of wait commands to direct a test script to pause for a certain time before throwing exception.
- In selenium, synchronization can be done using **4 wait commands**.

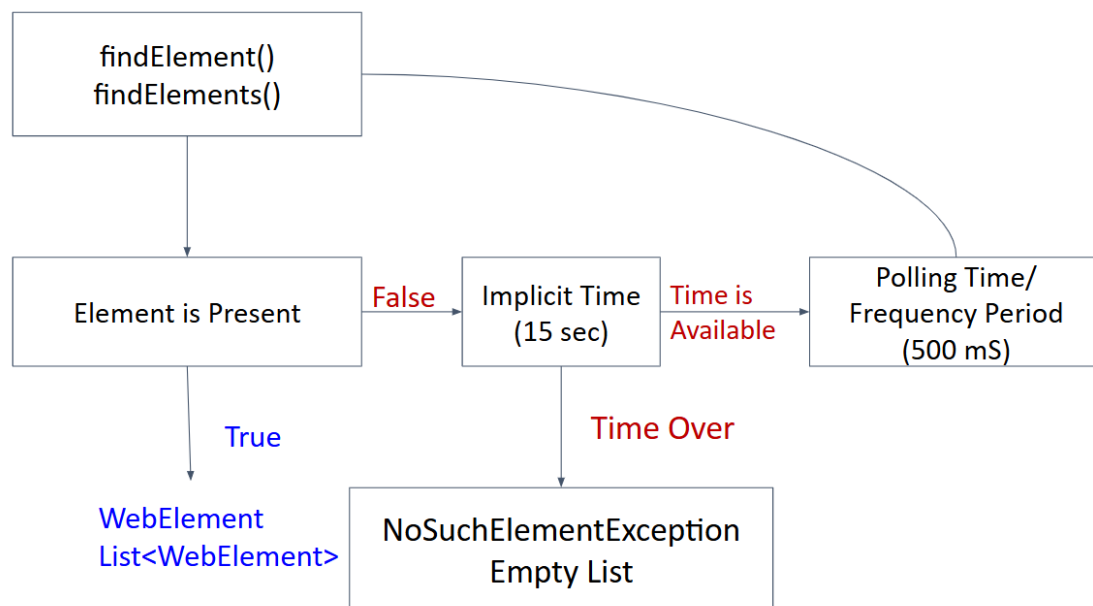
Types of Wait Commands :

1. Thread.sleep()
2. Implicit Wait
3. Explicit Wait
4. Fluent Wait

Note: We go for wait commands to reduce the usage of Thread.sleep()
[Static wait, Blind wait/ Dead wait/ Hardcoded wait]

1. Thread.sleep() :

- It is java wait statement which waits completely for the specified amount of time.
- It throws '**InterruptedException**'
- It accepts "time in milliseconds" as an argument.
- Disadvantage of Thread.sleep() :
- ✓ It is hard coded wait statement, where mandatorily the script is going to wait for the given seconds irrespective with the loading time of an application.



Activate Windows
Go to Settings to activate Windows.

2. Implicit Wait :

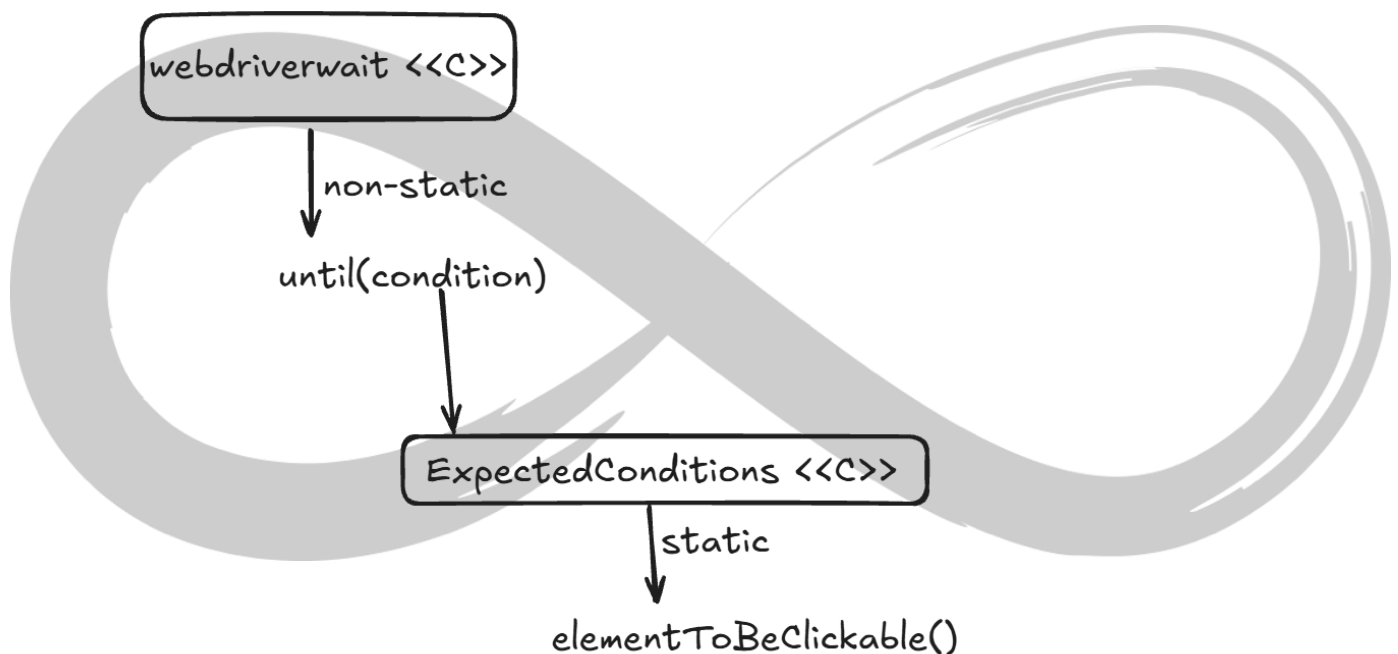
- It is selenium wait statement which is going to wait implicitly without specifying any condition
- It is 'Global wait statement' which means declare once and it will be holding for entire duration of the test
- It is applicable for each and every element on the webpage because it synchronizes for `findElement()` and `findElements()`
- Both implicit wait and Explicit wait can be used in same script, but there the first preference will be given to implicit wait and then goes to Explicit wait.
- It internally waits for particular period of time till the element is present, which is called as "Polling period"
- Syntax:
`driver.manage().timeouts().implicitlyWait(Duration.ofSeconds("time_in_seconds"));`

POLLING PERIOD :

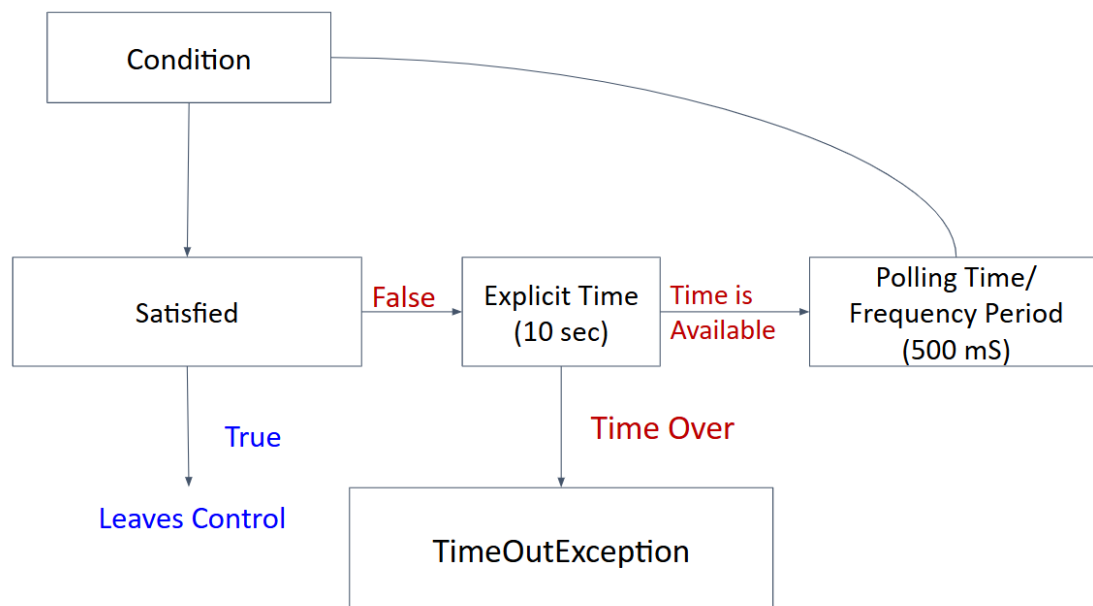
- It is the time interval in which selenium starts searching again for an element after it's last failed try.
- The default polling time is 500ms or 0.5sec

3. Explicit Wait (Smart wait):

- In Explicit wait you have to specify a waiting condition and it is going to wait only for that condition
- Explicit wait statement should be specified every time whenever synchronization is needed
- It can be achieved by 'WebDriverWait' Class
- It internally waits for particular period of time till the condition is satisfied, which is called as "Polling period"
- If the condition is not satisfied in the specified time period, it throws 'TimeoutException'.
- **Syntax :** WebDriverWait wait = new WebDriverWait(driver,TimeOut);



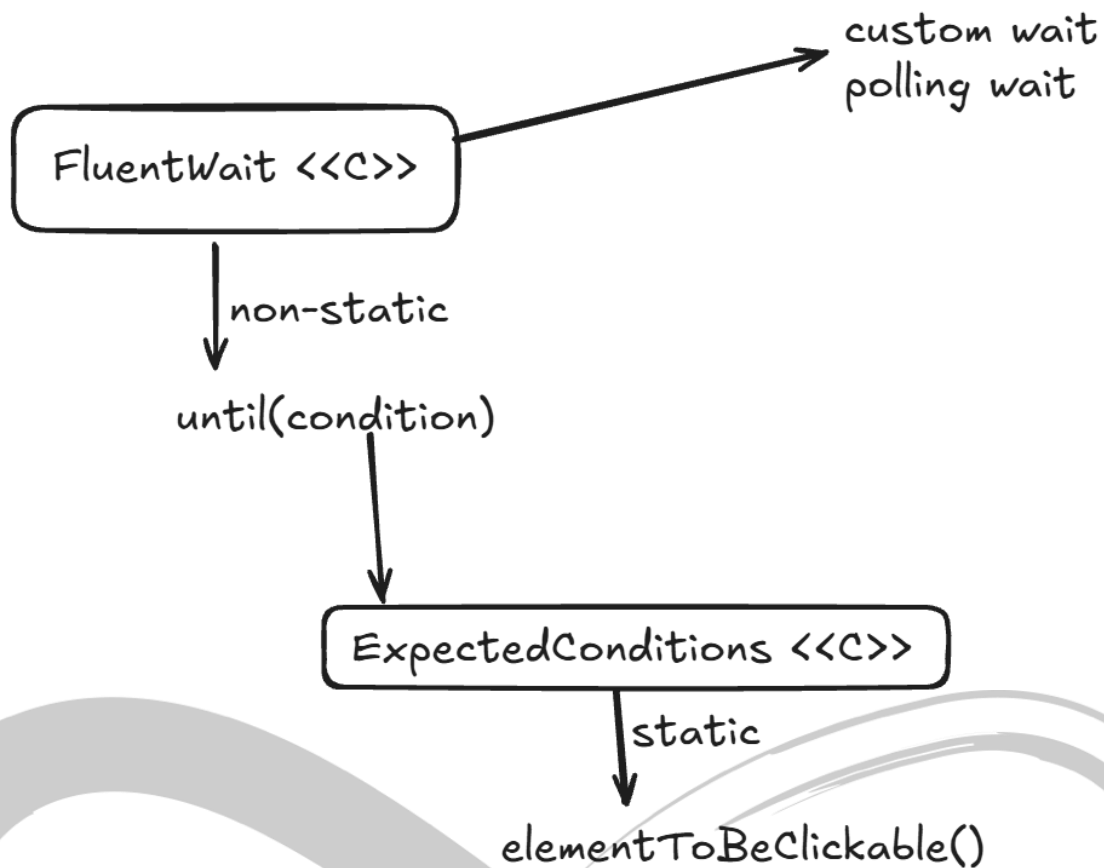
```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));  
wait.until(ExpectedConditions.elementToBeClickable(loginBtn))
```



Activate Windows
Go to Settings to activate Windows.

Methods present inside ExpectedConditions class: -

1. alertIsPresent()
2. elementSelectionModeToBe()
3. elementToBeClickable()
4. elementToBeSelected()
5. frameToBeAvaliableAndSwitchToIt()
6. invisibilityOfTheElementLocated()
7. invisibilityOfElementWithText()
8. presenceOfAllElementsLocatedBy()
9. presenceOfElementLocated()
10. textToBePresentInElement()
11. textToBePresentInElementLocated()
12. textToBePresentInElementValue()
13. titleIs()
14. titleContains()
15. visibilityOf()
16. visibilityOfAllElements()
17. visibilityOfAllElementsLocatedBy()
18. visibilityOfElementLocated()



```
FluentWait wait = new FluentWait(driver);
wait.withTimeout(Duration.ofSeconds(15));
wait.pollingEvery(Duration.ofMillis(1000))
wait.until(ExpectedConditions.elementToBeClickable(element))
```

4. Fluent Wait(Custom Wait, Polling Wait):

- Fluent wait is same as Explicit wait.
- Fluent wait statement should be specified every time whenever synchronization is needed
- Fluent Wait is used to customize the polling period.
- It ignores any exception that might occur before timeouts.
- It is not widely used in Industry.

Interview Questions:

1. What is synchronization?
2. How to handle sync?
3. Waiting commands available in Selenium WebDriver:
4. Difference between hardcoded wait, implicit wait and explicit wait
5. What is fluent wait? Explain it.
6. Which wait statement will be used to wait until the page loads?

Implicit Wait	Explicit Wait
It Works for findElement() and findElements()	It works for the given condition
If the element is not present, we get- NoSuchElementException or Empty List.	If the condition is not satisfied, we get TimeoutException

DROPDOWN/LISTBOX

Dropdown :

- It is a web element where we will select one value from the list of options dropped when clicked on it.
- There are 2 types of dropdowns/list boxes
 1. Single Select
 2. Multi Select
- Dropdowns are mostly developed using “ <select> ”, which means that can be handled using select class. And Contents of the drop down will be created using Option tag.
- Select class has parameterized constructor, where it takes an argument of type web element (Address of the Dropdown)
- There are 3 ways to handle dropdown-
 1. findElement() and findElements()
 2. By using Robot Class
 3. **By using Select Class**

Select Class:

- Select class provides the implementation of the HTML select tag
- It provides helper methods with Select and Deselect options
- It can be achieved by creating an object of the select class
- It also specifies the location of the web element
- Syntax: `Select sel = new Select(WebElement Address);`
 - with the help of reference variable, we can call 11 methods.

Select class Methods :

1. `selectByIndex(int index) :`
It is used to select the option based on index. Return type is “void”
2. `selectByValue(String value) :`
It is used to select the option based on value. Return type is “void”
3. `selectByVisibleText(String text) :`
It is used to select the option based on Visible Text. Return type is “void”
4. `getOptions() :`
It is used to get all the options from the Dropdown. Return type is “List<WebElement>”

5. `isMultiple()` :
→ It is used to check if dropdown is single select or multi select. Return type is “boolean”
6. `deselectByIndex(int index)` :
→ It is used to deselect the option based on index. Return type is “void”
7. `deselectByValue(String value)` :
→ It is used to deselect the option based on value. Return type is “void”
8. `deselectByVisibleText(String text)` :
→ It is used to deselect the option based on Visible Text. Return type is “void”
9. `deselectAll()` :
→ It is used to deselect all the selected options. Return type is “void”
10. `getFirstSelectedOption()` :
→ The first selected option in this select tag (or the currently selected option in a normal select) Return type is “WebElement”
11. `getAllSelectedOptions()` :
→ It is used to get all the selected options from the Dropdown.
Return type is “List<WebElement>”

Interview Questions:

1. How to Handle Dropdown?
2. List out all the methods available in select class.
3. How to capture all the values from the dropdown?
4. How do you handle dynamic google-search-style auto suggestion?
(Next goal : Auto-Suggestion)

AUTOSUGGESTIONS

- Auto suggestions dropdown is when you enter partial text in the search text field, gives the list of all the suggestions related to the text.
- We can fetch all the suggestions and can perform actions on the particular suggestion
- If the suggestion dropdown is not inspectable, follow these steps:
 1. Inspect the text field where we enter the partial text.
 2. Go to “Sources” tab in DOM
 3. Click on Pause Button **Once the suggestion dropdown appears.**
 4. Now, you can inspect the dynamic suggestion drop down with Ctrl + Shift + C



ACTIONS CLASS

- Actions Class is the concrete class of selenium. Which is used to perform various keyboard and mouse gestures like moving the cursor to an element or dragging and dropping an element on to the destination element.
- This class has approximately 22 methods to perform the keyboard and mouse gestures. They are:

Mouse Actions

- A. Hover
 1. moveToElement(element)
 2. moveByOffset(x, y)
 3. moveToElement(element, x, y)
- B. Left-click
 4. click()
 5. click(element)
- C. Right-click
 6. contextClick()
 7. contextClick(element)
- D. Double-click
 8. doubleClick()
 9. doubleClick(element)
- E. Click&Hold
 10. clickAndHold()
 11. clickAndHold(element)
- F. Release
 12. release()
 13. release(element)
- G. Drag&Drop
 14. dragAndDrop(src, dest)
 15. dragAndDropBy(src,x,y)

Scrolling Actions

- 16. scrollToElement(element)
- 17. scrollByAmount(x,y)

KeyBoard Actions

- 18. sendkeys()
- 19. keyDown()
- 20. keyUp()

Mandatory Actions

- 21. build()
- 22. perform()

How to use Actions Class?

1. Create an object of Actions Class and pass the **driver reference** to the constructor of the class.
2. Store the address of the object into a reference variable.

Eg: Actions action = new

Actions(driver)

3. Use the reference variable to call all the non-static methods of actions class.
4. Use method chaining to perform composite action on an element.

Eg: action.click(webelement).perform();

5. **Use perform method**, otherwise the action will not be performed.

ActionClass methods in Details:

1. click():

- In the series of composite actions, at some point we need to click on an element than we can use this method.
- Or else it will click on a location where current mouse pointer is available.

Signature: public Actions click(){

}

Usage: action.click(){

}

2. click(WebElement element):

- **In the series of composite actions to click on a WebElement we can use this method**

Signature: public Actions click(WebElement element){

}

Usage: action.click(element){

}

3. moveToElement(WebElement element):

- This method is used to handle dropdown menu or the menu which will open when you do mouse hovering. Using this method, we can hover pointer on top of a given target element.
- It will move the mouse pointer to the mid-point of the given target element.
- **Signature:** public Actions moveToElement(WebElement element){

}

Usage: action.moveToElement(target).perform();

4. moveToElement(WebElement target, xOffset, yOffset):

- This method is used to move the mouse pointer to the particular coordinate of the given target element.

For eg: To rate the food in Zomato we can move from the particular coordinate to the rating element and provide ratings.

- Here Xoffset and Yoffset means the x and y coordinates of the element.
- To move to the right side of the element we should give x value (for example 100) and y value should be 0, so that from the midpoint of that element mouse will move to 100 pixels right.

Signature:

```
public Actions moveToElement(WebElement element, int Xoffset,
intYoffset){
}
```

Usage: action.moveToElement(target, 100, 0);

5. moveByOffset(Xoffset, Yoffset):

- This method is used to move the cursor to a particular coordinate of the webpage.
- If we want to do double click or a right click on the same coordinate on the webpage we can use this method.

Signature:

```
public Actions moveByOffset(int Xoffset, intYoffset){
}
```

Usage: action.moveByOffset(200, 10).perform();

6. contextClick():

- It is used to perform right click on the location where the mouse pointer is available.
- The menu which will open is called as context menu.
- Whenever we have some operation to be done using context menu then we can use this method.

For example: Some of the application will have their own context menu in this case it will be useful.

Signature: public Actions contextClick(){
}

Usage: action. contextClick (){
}

7. contextClick(WebElement element):

- It is used to right click on any web-element

Signature: public Action contextClick(WebElement element){
}

Usage: action.contextClick(target);

8. clickAndHold():

- This method is used to click and hold the mouse cursor on the mouse pointer on the current mouse location.

- **Signature:** public Actions clickAndHold(){

}

- **Usage:** action.moveToElement(target).clickAndHold().perform();

9. clickAndHold(WebElement element):

- This method is used to click and hold the mouse cursor on the mouse pointer on the given element location.

- **Signature:** public Actions clickAndHold(webelement element){
}

- **Usage:** action. clickAndHold(element).perform();

10. release():

- This method is used to perform the release of mouse event.

- Whenever we need to drag and drop then we can use this method

- **Signature:** public Actions release(){
}

- **Usage:** action. release().perform();

11. release(WebElement element):

- This method is used to release the mouse keypress event.

- While performing clickAndHold option we can use this method.

➤ **Signature:** public Actions release(WebElement element){
}

➤ **Usage:** action. release(WebElement element).perform();

12. dragAndDrop(WebElement source, WebElement Destination):

➤ This method is used to place an element from one location to another location. Here, location is dependent on the given web element. To test such scenarios, we will use this method.

➤ **Signature:**

public Action dragAndDrop(WebElement source, WebElement Destination){

}

➤ **Usage:** action. dragAndDrop(source, destination).perform();

13. dragAndDropBy(WebElement element, intXoffset, intYoffset):

➤ This method is used to move the given element to a given coordinate of the webpage

➤ **Signature:**

public Action dragAndDrop(WebElement element, intXoffset, intYoffset){

}

➤ **Usage:** action.dragAndDropBy(target, 100, 50).perform();

14. doubleClick():

➤ This method is used to doubleClick on the location where mouse is available.

➤ **Signature:** public Actions doubleClick(){
}

➤ **Usage:** action. moveToElement(target).doubleClick ().perform();

15. doubleClick(WenElement target):

➤ This method is used to doubleClick on on a particular WebElement

➤ **Signature:** public Actions doubleClick(WebElement element){
}

➤ **Usage:** action.doubleClick (WebElement element).perform();

16. sendKeys(Character Sequence... args):

- This method is used to perform keyboard actions on the webpage.
- **Signature:** public Actions sendKeys(Character Sequence... keys){
}

Usage: action. sendKeys(Keys.PAGE_DOWN).perform();

17. keyDown(Character Sequence):

- This method is used to perform key press event
- This method can be used on three modifier keys i.e. Keys.CONTROL, Keys.SHIFT, Keys.ALT.

(Modifier means the key which modifies the functionality of other keys. For Example: Control + C □ To copy)

- Whenever we need to perform a control click operation, we can use this method.
- After calling this method key will remain in the presses state. So, KeyUP() method should be called to release the specific key.
- **Signature:** public Actions KeyDown(Character Sequence){
}
- **Usage:** action. KeyDown(Keys.CONTROL).perform();

18. keyUp(Character Sequence):

- This method is used to perform key release event. This method is opposite to keyDown() method.
- **Signature:** public Actions KeyUp(Character Sequence){
}
- **Usage:** action. KeyUp(Keys.CONTROL).perform();

19. build()

- This method is used to build all the composite actions which are stored.
- In the earlier versions of selenium before executing all the actions using perform method we had to call build method explicitly.
- In the latest versions of selenium perform method itself internally call build method, to build all the composite actions and later execute it. So no need to call build method explicitly.
- This method is the only method of Actions Class whose return type is Action.

➤ **Usage:** `action.click().build().perform();`

20. `perform()`:

- This method is used to execute single action or composite actions.
- If this method is not called, none of the actions will be executed.

Signature:

```
public Actions perform(){  
  
}
```

Usage: `action.moveToElement(target).perform();`



TAKESSCREENSHOT

- It is an interface which provides the mechanism to take screenshot.
- It has 2 different implementations
 1. RemoteWebDriver(for entire web page)
 2. RemoteWebElement (for a web element)
- It provides 1 abstract method
 - ❖ `getScreenshotAs()`
- This method take “OutputType” as an argument.
- We can store screenshot in 3 formats
 1. File
 2. Bytes
 3. Base64
- Return type is based on the argument passed

Eg:-

1. `getScreenshotAs(OutputType.FILE)` : File
2. `getScreenshotAs(OutputType.BYTES)` : Bytes
3. `getScreenshotAs(OutputType.BASE64)` : Base64/ String

Here are some typical cases where its crucial to take screenshot in Selenium:

- When application issues occur
- When an assertion failure occurs
- When there is some difficulty in finding web elements on a page
- Where there is a Timeout in finding web elements on a web page

HOW TO TAKE A SCREENSHOT

- `getScreenshotAs()` will take the screenshot and store it in a temporary directory and this screenshot will be deleted after the program ends.
- So we should move the file to a different location before the program ends.
- To move the file to a different location, we use `copy()` from `FileHandler` class.

1. To take a screenshot of a web page

```
TakesScreenshot tks= (TakesScreenshot) driver;  
File src= tks.getScreenshotAs(OutputType.FILE);  
File dest= new File("./folder-name/ss-name.extension");  
FileHandler.copy(src, dest);
```

2. To take a full screenshot of an entire web page (only for firefox)

```
HasFullPageScreenshot hps = (HasFullPageScreenshot) driver;  
File src =hps.getFullPageScreenshotAs(OutputType.FILE);  
File dest= new File("./folder-name/ss-name.extension");  
FileHandler.copy(src, dest);
```

3. To take a screenshot of a web element

```
WebElement element = driver.findElement(locator);  
File src= element.getScreenshotAs(OutputType.FILE);  
File dest= new File("./folder-name/ss-name.extension");  
FileHandler.copy(src, dest);
```

JAVASCRIPTEXECUTOR

Q. Why we should use javascript code in selenium WebDriver?

Ans:

- Sometimes selenium webdriver commands will not work as expected.

For example:

- click() may not work on the identified element. So in that case we can use java script code as a work-around.
- And also whenever there is no direct method to perform some operation on the webpage, we can use JavaScript code in selenium webdriver.

- JavascriptExecutor is an Interface, which provides the mechanism to write the Javascript code in Selenium Webdriver.
- In case, findElement(), findElements() and locators are not working, we can use JavascriptExecutor to perform an action on a web element.
- It provides 2 abstract methods:

1. executeScript("javascript code")
2. executeAsyncScript ("javascript code")

Synchronous means one task at a time or single threaded.

Asynchronous means multitask at a time or multiple threaded task.

- These 2 methods are implemented in RemoteWebDriver class.
- To use these methods, we should cast the WebDriver object to JavascriptExecutor type.

Syntax:-

```
JavascriptExecutor js= (JavascriptExecutor) driver;
```

```
js.executeScript ("javascript code");
```

❑ **Notes:**

executeScript() receives 2 inputs

1. **Javascript code** in String format.
2. **Generic type** which acts as **input** to Javascript code.

In JavaScript, by default **3 global objects** will be available.

Using these global objects, we can invoke functions in JavaScript.

1. **Window object** :- represented by reference variable, window.
2. **Document object** :- represented by reference variable, document.
3. **Arguments[0]** :- global variable

Operations performed by using JavascriptExecutor

1. Open the website
2. Window scrolling
3. Type some text
4. Click operation
5. Performing action on disabled element
6. Performing action on hidden element

1. Open the Website

```
js.executeScript("window.location='URL'");
```

2. Window Scrolling

To scroll, there are 3 ways:-

(i) `scrollBy(int x, int y)`

This method is used to scroll the window horizontally or vertically based on x and y coordinates.

(ii) `scrollTo(int x, int y)`

This method is used to scroll the window to a particular coordinate from Origin.

(iii) `scrollIntoView(Boolean)`

This method is used to move the element to the visible area of the page.

Syntax:-

```
js.executeScript("window.scrollBy(int x, int y)"); // Relative Scrolling  
js.executeScript("window.scrollTo(int x, int y)"); // Absolute Scrolling  
js.executeScript("arguments[0].scrollIntoView(Boolean)", WebElement);  
//Till that particular element
```

Note:

- To scroll the window to **extreme bottom** of the web page:-
`window.scrollTo(0, document.body.scrollHeight);`
- To scroll to **extreme top** of the web page:-
`Window.scrollTo(0, -document.body.scrollHeight);`

```
String fqp = "https://www.facebook.com/";  
String jsCode = "window.location='" + fqp + "';";
```

```
System.out.println(jsCode);
```

Result:

```
window.location='https://www.facebook.com/';
```

```
String fb = "https://www.facebook.com/";  
String insta = "https://www.instagram.com/";  
String zom = "https://www.zomato.com/";
```

```
jse.executeScript("window.location=arguments[0];",fb, insta, zom);
```

```
jse.executeScript("window.location=arguments[1];",fb, insta, zom);
```

```
jse.executeScript("window.location=arguments[2];",fb, insta, zom);
```

0	1	2
---	---	---

```
WebElement un = driver.findElement(By.id("email"));  
// Java-Selenium  
un.sendKeys("admin");
```

```
// Javascript-Selenium  
String username = "admin";
```

```
jse.executeScript("arguments[0].value = arguments[1]", un, username);
```

un	.value =	username;
		'admin'

3. Type some text

```
js.executeScript("arguments[0].value='Message'", element);
```

4. Click operation

```
js.executeScript("arguments[0].click()", WebElement);
```

5. Performing action on Disabled Element

- In real life we won't be performing action on disabled element, because the element is disabled for a reason.
- So until the condition is not satisfied we should never be able to perform any action on disabled element.

Syntax:-

```
WebElement we= driver.findElement(By.id("abc"));
```

```
js.executeScript("arguments[0].click() ", we);
```

6. Performing action on Hidden Element

- To handle hidden element, we use **arguments[0].value='value'**

Syntax:-

```
WebElement we= driver.findElement(By.id("abc"));
```

```
js.executeScript("arguments[0].value='message' ", we);
```

SHADOW DOM

- Shadow DOM is a web standard used to encapsulate the HTML, CSS, and JavaScript of a web component from the main DOM.
- It creates a separate DOM tree called the **Shadow Tree**, which is attached to a normal DOM element called the **Shadow Host**.
- Selenium 4 provides built-in support to interact with Shadow DOM elements.

Why was Shadow DOM introduced?

- To hide internal implementation details of web components.
- To avoid CSS and JavaScript conflicts.
- To create reusable and independent UI components.
- Used in modern frameworks and browsers.

Shadow DOM Terminologies

1. Shadow Host

- The normal HTML element that contains the Shadow DOM.

Example:

```
<div id="host"></div>
```

2. Shadow Root

- The root element of the Shadow DOM.

There are two types:

Open Shadow Root

Closed Shadow Root

3. Shadow Tree

- The complete DOM structure inside the Shadow Root.

Types of Shadow DOM

1. Open Shadow DOM

- Internal elements are accessible.
- Selenium can automate these elements.

2. Closed Shadow DOM

- Internal elements are hidden.
- Selenium cannot access them directly.

How Selenium Handles Shadow DOM

Before Selenium 4

- We had to use JavaScriptExecutor.

After Selenium 4

- Selenium introduced the **getShadowRoot()** method.

Step 1

Locate Shadow Host

```
WebElement shadowHost = driver.findElement(By.cssSelector("css"));
```

Step 2

Get Shadow Root

```
SearchContext shadowRoot = shadowHost.getShadowRoot();
```

Step 3

Locate element inside Shadow DOM

```
shadowRoot.findElement(By.cssSelector("input"));
```

Nested Shadow DOM

Sometimes one Shadow DOM contains another Shadow DOM.

```
WebElement host1 = driver.findElement(By.cssSelector("host1"));
```

```
SearchContext root1 = host1.getShadowRoot();
```

```
WebElement host2 = root1.findElement(By.cssSelector("host2"));
```

```
SearchContext root2 = host2.getShadowRoot();
```

```
root2.findElement(By.cssSelector("input")).sendKeys("Hello");
```

Notes:-

- Closed Shadow DOM cannot be automated directly.
- Requires Selenium 4 or JavaScriptExecutor.
- Locators outside Shadow DOM cannot directly identify elements inside it.

Interview Questions

Q1. What is Shadow DOM?

Ans:

Shadow DOM is a separate DOM tree attached to a normal HTML element that encapsulates HTML, CSS, and JavaScript from the main DOM.

Q2. What is the difference between DOM and Shadow DOM?

DOM	Shadow DOM
Visible in Main DOM	Separate DOM Tree
Accessible directly	Access through Shadow Root
CSS can affect all elements	CSS is encapsulated

Q3. What is Shadow Host?

Ans:

The HTML element to which the Shadow DOM is attached is called the Shadow Host.

Q4. What is Shadow Root?

Ans:

The root node of the Shadow DOM. Selenium accesses internal elements through the Shadow Root.

Q5. What are the types of Shadow DOM?

Ans:

Open Shadow DOM

Closed Shadow DOM

Q6. Can Selenium automate Closed Shadow DOM?

Ans:

No. Selenium cannot directly access elements inside a Closed Shadow DOM because the Shadow Root is hidden.

Q7. Which method is used to access Shadow DOM in Selenium 4?

Ans:

`getShadowRoot()`

Q8. Which interface is returned by `getShadowRoot()`?

Ans:

`SearchContext`

IFRAMES

- A web page inside another web page is called as embedded web page(frames).
- Embedded web pages are created using the “iframe” tag.
- **Purpose of iframe:**
- Advertisement
- Commonly used features(captcha)
- Common Page(If two or more developers working on a same page)

Syntax :- driver.switchTo().frame();

- frame() is overloaded, we can switch driver control from parent window to frame in 3 ways

1. frame(int index)
2. frame(String name/id)
3. frame(WebElement ele)

Note:-

To switch back the driver control from frame to parent window, we have 2 methods

1. driver.switchTo().parentFrame()
- To switch back the driver control to immediate parent frame.
2. driver.switchTo().defaultContent()
- To switch back the driver control to main page.

POP-UPS

- It is a GUI window which opens on top of the browser window.
- In automation we should take care of testing the popups, like verifying the expected behaviors of the popups are working fine or not, pop ups texts are working as per the requirement or not.

Purpose of PopUps:

1. To give information to end-users.
2. To get information from end-users.
3. To grab attention of end-users. (especially the Ads).

What are the strategies to handle a popup?

1. Check whether we can inspect the popup or not. If we can inspect then we can Handle the pop up using `findelement()`, `findelements()` and `actions` class etc.
2. If the pop up cannot be inspected then we should investigate whether any library method is provided within selenium webdriver. If provided use those to handle the popup.
3. If not then try to avoid the popup.
4. Use robot class methods to handle the popup.
5. Use open-source third party tools, one of them being **AutoIT**.

Popups are categorized into following types:

1. JavaScript Pop-up
2. Notification popup
3. Authentication popup
4. Hidden division Pop-ups/Modals/Overlays
5. File download popup
6. File upload Pop-up
7. Child Window popup (Window Handling)

1. JavaScript Pop-up

- These popups are developed using JavaScript language

Types of JavaScript popups

1. Alert popup
2. Confirmation popup
3. Prompt popup

Characteristics of JavaScript popups

- 1) We cannot inspect this popup
- 2) We cannot move this popup
- 3) We cannot avoid these popups
- 4) We cannot perform any action on the web page unless you handle the popup.

➤ **Alert popup (**

This popup will have OK button

➤ **Confirmation popup**

This popup contains OK and CANCEL Button

➤ **Prompt popup**

This popup contains Text field.

To Handle Javascript Popup

Syntax:- driver.switchTo().alert();

1. **accept()** :- To click on the OK button.
2. **dismiss()**:- To click on the CANCEL button.
3. **getText()**:- To get the text present on the popup.
4. **sendKeys()**:- To enter the text on the popup.

Note:-

- Without handling JavaScript popup, when we try to perform tasks, we will get “**UnhandledAlertException**”,.
- If we try to switch driver control to alert popup before it appears, we will get “**NoAlertPresentException**”

2. Notification Popups

Characteristics:-

1. We cannot move this popup.
2. We cannot inspect this popup.
3. We can avoid and handle this popup.
4. This popup has an allow and block button.
5. It will be displayed below the address bar in the beginning.

To Avoid Notification Popups:-

- In order to avoid this popup, we can change the settings of the browser. So that notification popup will not be displayed.
- To change the settings of the browser, we use **addArguments()** of **ChromeOptions** class.

How to Avoid:-

- Create an object of ChromeOptions and store it in a reference variable.
- Using this reference variable, call the method addArguments() and pass Chromium commands.
- While launching the browser, pass the reference variable of ChromeOptions in the constructor call.

Note:-

chromeOptions is for chrome Browser.

For edge : edgeOptions

For firefox : firefoxOptions, etc.

Syntax:-

```
ChromeOptions settings= new ChromeOptions();  
settings.addArguments("- -disable-notifications");  
Also we are having...  
settings.addArguments("- -incognito");  
settings.addArguments("--start-maximized");
```

Assignment

- Navigate to “makemytrip.com”
- Avoid Notification Popup
- Navigate to “yatra.com”
- Avoid Notification Popup
- Navigate to “easemytrip.com”
- Avoid Notification Popup

3. Authentication Popups

Characteristics:-

1. We cannot move this popup
2. We cannot inspect this popup
3. We can avoid and handle this popup
4. This popup will have a username and password textbox along with sign-in and cancel button.

To Handle Authentication Popups:-

We can handle this popup with the help of Robot Class.

To Avoid Authentication Popups:-

- We can avoid this popup by sending username and password along with the URL inside get().

Syntax:- driver.get("https://UN:PWD@rest of URL");

Assignment

- Navigate to <https://httpbin.org/basic-auth/user/pass>
- Avoid authentication pop-up using un : user and pwd : pass

4. Hidden Division Popups/ Modals/ Overlays

- The popup which is created using the html code is called as the hidden division popup.
- This popup will appear anywhere on the webpage.

Characteristics:-

1. We cannot move this popup
2. We can inspect this popup
3. We cannot avoid this we need to handle it.

→ Calendar popup is the best example

To handle Hidden Division Popups:-

To Handle it, we can use 2 methods.

1. findElement()
2. findElements()

Generally we make use of findElement().

Assignment

- Navigate to makemytrip.com
- Select "any date" eg: 03-april, 26-Jan-2025,

5. File Download Popups

By default, this popup will be avoided by the Selenium.

6. File Upload Popups

Characteristics:-

- 1) We can move this popup
- 2) We cannot inspect this popup
- 3) We can either avoid or handle file upload popup
- 4) This popup will have open and cancel button

To avoid file upload popup:-

- 1) Inspect the element.
- 2) Search for Input tag
- 3) Find locator strategy and write fully qualified file path.
- 4) Open the location where the file to be uploaded is present and copy the path. And then make use of sendKeys(“absolute path of file”).

To Handle File Upload popups:-

- To handle, we have to use Robot Class
- We can handle by using 3rd party library called AutoIT.

7. Child Window Popups (Handling Windows)

Sometimes when the user opens a webpage along with the webpage or when he clicks on a link, another browser window opens, this browser window is known as child browser or child browser popup.

Characteristics:-

- 1) We can move the popup
- 2) We can inspect the popup
- 3) This popup has minimize, maximize and close buttons along with the address bar.

To handle child window popups:-

We can make use of –

- a) getWindowHandle()
- b) getWindowHandles()
- c) switchTo()

Steps to handle multiple windows:-

step 1> get the home address

```
String PID = driver.getWindowHandle();  
32 digits unique session id in string format
```

step 2> perform the task which will open new window/s or tab/s

```
driver.findElement(By.LinkText("Meta Pay")).click();
```

Step 3> once all the windows are opened, get their addresses

```
Set<String> CIDs = driver.getWindowHandles();  
returns Set<String> of multiple unique session ids
```

step 4> go through each and every window

```
for (String i : CIDs) {  
    driver.switchTo().window(i);  
    String title = driver.getTitle();
```

step 5> give the condition for your desired webpage and break the loop

```
    if(title.contains("Pay")) {  
        break;  
    }  
}
```

step 6> perform the task on your desired webpage

```
Thread.sleep(2000);  
driver.close();
```

step 7> come back to home

```
driver.switchTo().window(PID);
```

New Window / Tab Handling in Selenium 4

```
driver.switchTo().newWindow(WindowType.TAB);
```

EXCEPTIONS

No.	Exception	Reason
1.	StaleElementReferenceException	Web element reference becomes invalid because the DOM has changed, page refreshed
2.	NoSuchElementException	Selenium cannot find the element using the provided locator.
3.	ElementNotInteractableException	Element exists in DOM but is hidden, disabled, or not ready for interaction.
4.	ElementClickInterceptedException	Another element (popup, loader, overlay, banner) intercepts the click action.
5.	TimeoutException	A wait condition is not satisfied within the specified timeout period.
6.	SessionNotCreatedException	Browser session cannot be created, usually due to browser-driver version mismatch.
7.	NoSuchWindowException	Selenium tries to switch to a window that does not exist or has been closed.
8.	NoSuchFrameException	Selenium cannot locate the specified frame or iframe.
9.	InvalidSelectorException	XPath, CSS Selector, or locator syntax is invalid.
10.	JavascriptException	JavaScript execution fails due to syntax errors or invalid script execution.
11.	MoveTargetOutOfBoundsException	Actions class tries to move the mouse to an element outside the browser viewport.
12.	ElementNotSelectableException	Selenium attempts to select an element that is not selectable.
13.	NoAlertPresentException	Selenium tries to handle an alert that is not currently displayed.
14.	UnhandledAlertException	An unexpected alert is present and blocks further WebDriver operations.
15.	InvalidArgumentException	An invalid argument is passed to a WebDriver command or method.
16.	NoSuchSessionException	WebDriver tries to use a browser session that has already been closed or quit.
17.	WebDriverException	Generic Selenium exception indicating a WebDriver-related failure not covered by a specific exception.
18.	UnsupportedCommandException	The browser driver does not support the requested WebDriver command.